

Cross Segmentos - Linha Datasul - MLA - Criação da API para disponibilizar documentos para o APP

 Tempo aproximado para leitura: 00:11:20 min

Dúvida

Como realizar a criação da **API** para disponibilizar documentos para o **Aplicativo - APP Minhas Pendências de Aprovação** do **MLA - Módulo Logístico de Aprovação**?

Ambiente

Cross Segmentos - TOTVS Backoffice (Linha Datasul) - Aprovação Processo Logístico (MLA) - Versão 12

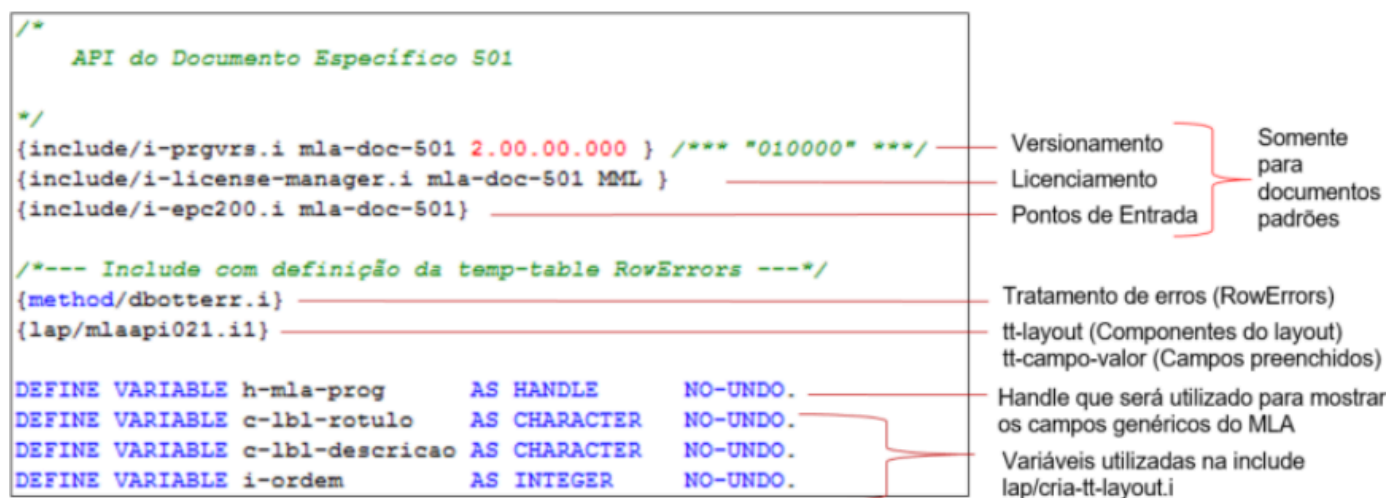
Solução

A nomenclatura da API a ser criada deve ser **lap/mla-doc-<número doc>.p**.
Por exemplo:

- Para o documento **13** seria **lap/mla-doc-13.p**.
- Para o documento **501** seria **lap/mla-doc-501.p**.

É importante entender que este programa será utilizado para configurar os campos que devem aparecer na listagem e detalhamento das pendências no **APP**, assim como para preencher os campos com os seus respectivos valores.

Para o cabeçalho segue exemplo, com explicações:



```
/*
  API do Documento Específico 501
*/
{include/i-prgvrs.i mla-doc-501 2.00.00.000 } /** "010000" ***/
{include/i-license-manager.i mla-doc-501 MML }
{include/i-epc200.i mla-doc-501}

/*--- Include com definição da temp-table RowErrors ---*/
{method/dbotterr.i}
{lap/mlaapi021.i1}

DEFINE VARIABLE h-mla-prog          AS HANDLE          NO-UNDO.
DEFINE VARIABLE c-lbl-rotulo        AS CHARACTER       NO-UNDO.
DEFINE VARIABLE c-lbl-descricao     AS CHARACTER       NO-UNDO.
DEFINE VARIABLE i-ordem             AS INTEGER         NO-UNDO.
```

Versionamento
Licenciamento
Pontos de Entrada

Somente para documentos padrões

Tratamento de erros (RowErrors)
tt-layout (Componentes do layout)
tt-campo-valor (Campos preenchidos)

Handle que será utilizado para mostrar os campos genéricos do MLA

Variáveis utilizadas na include lap/cria-tt-layout.i

Para documentos específicos **não** é necessário a parte de **versionamento**, **licenciamento** e **pontos de entrada**.

A **include** **lap/mlaapi021.i1** e **lap/cria-tt-layout.i** são **liberadas** para que se possa fazer a compilação dos programas. Na sequência, deve-se implementar as **procedures** **pi-retorna-campos-lista** e **pi-retorna-campos-detalle**.

- **pi-retorna-campos-lista**: Utilizada para retorno dos campos que podem ser configurados para a listagem
- **pi-retorna-campos-detalle**: Utilizada para retorno dos campos que podem ser configurados para o detalhe

As duas *procedures* possuem a mesma estrutura de parâmetros de **entrada** e **saída**.

Entrada/Saída	Nome	Tipo	Nome
Entrada	cUsuario	<i>Character</i>	Usuário para o qual se deseja configurar o <i>layout</i>
Entrada	iDocumento	<i>Integer</i>	Documento para o qual se deseja configurar o <i>layout</i>
Entrada	iInterface	<i>Integer</i>	Interface para a qual se deseja configurar o <i>layout</i> 1 - Lista / 2 - Detalhe
Entrada	ISomenteVisiveis	<i>Logical</i>	Indica que a <i>procedure</i> deve retornar apenas os campos visíveis, se não marcado, deve retornar todos
Saída	tt-layout	<i>temp-table</i>	<i>Temp-table</i> contendo os campos para o <i>layout</i>

Sobre a **estrutura** da *temp-table* *tt-layout*:

tt-layout		
Campo	Tipo	Descrição
cdn-docto	<i>Integer</i>	Código do Documento

tt-layout		
Campo	Tipo	Descrição
cod-usuar	<i>Character</i>	Código do Usuário
idi-interfac	<i>Integer</i>	Tipo de Interface
cod-id-campo	<i>Character</i>	Id do Campo
idi-compon	<i>Integer</i>	Tipo de Componente 1 – Campo 2 – Lista 3 - Separador
nom-rotu-campo	<i>Character</i>	Rótulo do componente
des-campo	<i>Character</i>	Descrição do componente
log-visivel	<i>Logical</i>	Indicador se o campo está visível
cdn-tam	<i>Integer</i>	Tamanho do componente
cdn-ord	<i>Integer</i>	Ordem do componente
cod-entid-pai	<i>Character</i>	ID do componente pai

Para a criação dos campos na *temp-table*, sugere-se a utilização da *include* **lap/cria-tt-layout.i**, pois ela já faz o tratamento para **técnica de tradução** e também já faz o tratamento para **criar** na *temp-table* os campos para retorno conforme visibilidade. Por exemplo, se estiver marcado para **retornar somente visíveis**, criará apenas os campos visíveis para retorno. Se fizer a criação

manualmente, precisará se preocupar com estes detalhes.

Exemplo de utilização abaixo, lembrando que deverá existir uma chamada para cada componente:

```
{lap/cria-tt-layout.i &cCodIdCampo="doc-nr-requisicao"  
    &iIdiCompon="1"  
    &cNomRotuCampo="Requisição"  
    &cDesCampo="Número da Requisição"  
    &lLogVisivel="TRUE"  
    &iCdnTam="100"}
```

Se utilizada esta *include*, a ordenação dos campos, seguirá a ordem em que forem criados.

Para as *procedures* de retorno de campos para **lista** e **detalhe**, caso se deseje retornar também os campos genéricos do **MLA**, pode-se chamar o programa que os retorna, dentro de cada uma das *procedures*. Por exemplo, para incluir os campos de **informações genéricas** do **MLA** na **lista** ou **detalhe**, pode-se utilizar o trecho a seguir.

```
/* Informação genéricas do MLA */  
IF NOT VALID-HANDLE(h-mla-prog) THEN DO:  
    RUN lap/mla-doc-generico.p PERSISTENT SET h-mla-prog.  
END.  
  
EMPTY TEMP-TABLE tt-layout-modelo.  
  
RUN pi-retorna-campos-lista IN h-mla-prog (INPUT cUsuario,  
    INPUT iDocumento,  
    INPUT iInterface,  
    INPUT lSomenteVisiveis,  
    OUTPUT TABLE tt-layout-modelo) NO-ERROR.  
  
FOR EACH tt-layout-modelo:  
    IF NOT CAN-FIND(FIRST tt-layout WHERE tt-layout.cdn-docto = tt-layout-modelo.cdn-docto  
        AND tt-layout.cod-usuar = tt-layout-modelo.cod-usuar  
        AND tt-layout.idi-interfac = tt-layout-modelo.idi-interfac  
        AND tt-layout.cod-id-campo = tt-layout-modelo.cod-id-campo) THEN DO:  
  
        {lap/cria-tt-layout.i &cCodIdCampo="tt-layout-modelo.cod-id-campo"  
            &iIdiCompon="tt-layout-modelo.idi-compon"  
            &cNomRotuCampo="tt-layout-modelo.nom-rotu-campo"  
            &cDesCampo="tt-layout-modelo.des-campo"  
            &lLogVisivel="false"  
            &iCdnTam="100"  
            &cCodEntidPai="tt-layout-modelo.cod-entid-pai"}  
  
    END.  
END.
```

Veja que neste exemplo, na chamada na *include* **lap/cria-tt-layout.i**, foi fixado para que todos os campos genéricos fiquem como **não visíveis** - **&lLogVisivel="false"** e o tamanho como **100** - **&iCdnTam="100"**, se desejasse que eles tivessem a mesma visibilidade e tamanho que o padrão para os documentos genéricos, poderia utilizar **&lLogVisivel="tt-layout-modelo.log-visivel"** e **&iCdnTam="tt-layout-modelo.cdn-tam"**.

Para buscar os campos genéricos do **MLA** para o **detalhe**, deve-se utilizar a mesma estrutura

acima, apenas alterando a *procedure* para **pi-retorna-campos-detalle**.
Caso queira saber quais os campos genéricos do **MLA** para o **APP**, consultar o artigo Cross Segmentos - Linha Datasul - MLA - Campos genéricos do MLA no APP Minhas Pendências de Aprovação

Para o retorno da sigla do documento deve-se implementar a *procedure* **pi-retorna-sigla**. Com a seguinte estrutura de parâmetros a seguir:

Entrada/Saída	Nome	Tipo	Nome
Entrada	iDocumento	<i>Integer</i>	Código do documento
Saída	cSigla	<i>Character</i>	Sigla, contendo no máximo duas posições.

Exemplo:

```
PROCEDURE pi-retorna-sigla:
/*****
Retorna sigla do documento
iDocumento (INPUT - Integer): Documento para o qual se deseja configurar o layout
cSigla (OUTPUT - Character): Sigla do documento
*****/
  DEFINE INPUT  PARAM iDocumento      AS INTEGER    NO-UNDO.
  DEFINE OUTPUT PARAM cSigla          AS CHARACTER NO-UNDO.

  /* Sigla para o documento */
  ASSIGN cSigla = "ES":U.

  RETURN "OK":U.
END PROCEDURE.
```

É necessário também criar as *procedures* que irão fazer o **preenchimento dos valores** para mostrar cada campo nas pendências. Para facilitar as explicações, vamos utilizar um exemplo, de que o programa a ser criado deve contemplar a apresentação do documento abaixo:

Número: 10020
Descrição: Documento 10020
Situação: Aprovado
Responsável Principal: SUPER
Valor: 10,50
Narrativa: Documento específico para testes do APP MLA

Itens

Código: .compras
 Descrição: ITEM .compras

Entregas

Seq: 1 Data: 11/12/2020

Seq: 2 Data: 12/12/2020

Código: .compras2
 Descrição: .compras2 ITEM

Entregas

Seq: 1 Data: 01/12/2020

Seq: 2 Data: 10/12/2020

Seq: 3 Data: 01/01/2021

Deve-se então implementar as *procedure* **pi-preenche-lista** e **pi-preenche-detalle** que são utilizadas para o preenchimento dos campos da listagem e do detalhamento de pendências, respectivamente, com seus valores de negócio. A estrutura de parâmetros destas *procedures* deve ser a seguinte:

Entrada/Saída	Nome	Tipo	Nome
Entrada	iNrTransacao	<i>Integer</i>	Número da transação da pendências
Entrada/Saída	tt-campo-valor	<i>Temp-table</i>	IDs de componentes com seus respectivos valores. Observação: A <i>procedure</i> recebe uma lista de IDs de componentes e deve preencher a <i>temp-table</i> com os respectivos valores conforme regras de negócio.

Para a *procedure* **pi-preenche-lista**, temos o exemplo de implementação na sequência, com explicação dos códigos:

1. É necessário localizar as tabelas de negócio. Neste caso **documento** é uma *temp-table*, com somente um registro para demonstração, por isso o **FIND FIRST**. Na prática deve-se localizar a tabela de negócio conforme a chave do documento disponibilizada pela leitura da tabela **mldoc-pend-aprov**.

Abaixo um exemplo para localização da tabela de negócio com base na chave do documento. Definir a *include* **lap/mlaapi001.i99** no início do programa para ter disponível da *temp-table* **tt-mla-chave**, e as três variáveis abaixo:

```
/* Definição tt-mla-chave */
{lap/mlaapi001.i99}
/* Variáveis utilizadas pela include mlaapi001.i */
DEFINE VARIABLE c-chave      AS CHARACTER    NO-UNDO.
DEFINE VARIABLE i-chave      AS INTEGER      NO-UNDO.
DEFINE VARIABLE i-tipo-doc AS INTEGER        NO-UNDO.
```

Utilizar a *include* **laphtml/mlacriachave.i** para preencher a **tt-mla-chave** com os campos da chave do documento, neste caso são **três**. Após isso, utilizar cada posição para localizar o registro de negócio.

```
/* *****
Chave documento 1 - Solicitação de compra - Item
 1 a 9: Nr Solicitação
10 a 12: Seq Item
13 a 28: Cod Item
***** */

/* Cria a tt-mla-chave com a chave do documento */
{laphtml/mlacriachave.i iNrTransacao}

FIND FIRST it-requisicao
WHERE it-requisicao.nr-requisicao = INT(tt-mla-chave.valor[1])
AND it-requisicao.sequencia = INT(tt-mla-chave.valor[2])
AND it-requisicao.it-codigo = tt-mla-chave.valor[3] NO-LOCK NO-ERROR.
```

2. Preencher os registros da *temp-table* que vieram com **IDs** preenchidos com os respectivos valores. A informação deve ser sempre retornada como **STRING** e formatada como se deseja que apareça no **APP**.

3. Caso nas *procedures* de retorno de campos tenha definido para mostrar campos genéricos do **MLA** é necessário o preenchimento deles.

```

PROCEDURE pi-preenche-lista:
  DEFINE INPUT  PARAM iNrTransacao          AS INTEGER  NO-UNDO.
  DEFINE INPUT-OUTPUT PARAM TABLE FOR tt-campo-valor.

  IF NOT VALID-HANDLE(h-mla-prog) THEN DO:
    RUN lap/mla-doc-generico.p PERSISTENT SET h-mla-prog.
  END.

  FIND FIRST mla-doc-pend-aprov WHERE mla-doc-pend-aprov.nr-trans = iNrTransacao NO-LOCK NO-ERROR.
  FIND FIRST documento NO-LOCK NO-ERROR.

  FOR EACH tt-campo-valor EXCLUSIVE-LOCK:
    CASE tt-campo-valor.id-campo:
      WHEN "doc-numero":U THEN
        ASSIGN tt-campo-valor.valor-campo = STRING(documento.num-doc) .
      WHEN "doc-descricao":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.descricao.
      WHEN "doc-situacao":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.situacao.
      WHEN "doc-responsavel":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.responsavel.
      WHEN "doc-valor":U THEN
        ASSIGN tt-campo-valor.valor-campo = STRING(documento.valor, ">>>, >>>, >>9.99") .
      WHEN "doc-narrativa":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.narrativa.
    END CASE.
  END.

  RUN pi-preenche-lista IN h-mla-prog (INPUT iNrTransacao,
                                         INPUT-OUTPUT TABLE tt-campo-valor).

  RETURN "OK":U.
END PROCEDURE.

```

Na *procedure* **pi-preenche-lista** é importante não eliminar o *handle* das informações genéricas do **MLA** - **h-mla-prog**, pois ela será chamada várias vezes de forma persistente. Então deve-se criar a *procedure* **pi-finaliza** eliminando este *handle*.

```

PROCEDURE pi-finaliza:
/*****
  Elimina handles utilizados
*****/

  IF VALID-HANDLE(h-mla-prog) THEN DO:
    RUN pi-finaliza IN h-mla-prog.
    DELETE PROCEDURE h-mla-prog.
    ASSIGN h-mla-prog = ?.
  END.

  RETURN "OK":U.
END PROCEDURE.

```

Para a *procedure* **pi-preenche-detalle**, a estrutura inicial é muito semelhante a *procedure* de **lista**. A diferença que temos é para os **componentes de listas**, com a necessidade de definição de 5 variáveis para cada lista, que serão explicadas a utilização mais para frente. O valor do **extent** deve ser o número de componentes que a lista possui. Neste caso temos duas listas, de **itens** e de **entregas**, cada uma com dois campos, portanto os

extents tem o **valor 2**. É necessário fazer a leitura das tabelas de negócio, assim como explicado na lista.

```
PROCEDURE pi-preenche-detalhe:
/*****
Preenche os campos de um detalhamento com seus respectivos valores

iNrTransacao (INPUT - Integer): Número da transação da pendência
tt-campo-valor (INPUT-OUTPUT - Tem-table): IDs de campos e seus valores
*****/

DEFINE INPUT PARAM iNrTransacao AS INTEGER NO-UNDO.
DEFINE INPUT-OUTPUT PARAM TABLE FOR tt-campo-valor.

DEFINE VARIABLE l-existe-campo-itens AS LOGICAL NO-UNDO.
DEFINE VARIABLE l-cria-filho-itens AS LOGICAL EXTENT 2 NO-UNDO.
DEFINE VARIABLE c-campo-filho-itens AS CHARACTER EXTENT 2 NO-UNDO.
DEFINE VARIABLE i-cont-campos-itens AS INTEGER NO-UNDO.
DEFINE VARIABLE i-cont-itens AS INTEGER NO-UNDO.

DEFINE VARIABLE l-existe-campo-entregas AS LOGICAL NO-UNDO.
DEFINE VARIABLE l-cria-filho-entregas AS LOGICAL EXTENT 2 NO-UNDO.
DEFINE VARIABLE c-campo-filho-entregas AS CHARACTER EXTENT 2 NO-UNDO.
DEFINE VARIABLE i-cont-campos-entregas AS INTEGER NO-UNDO.
DEFINE VARIABLE i-cont-entregas AS INTEGER NO-UNDO.

IF NOT VALID-HANDLE(h-mla-prog) THEN DO:
    RUN lap/mla-doc-generico.p PERSISTENT SET h-mla-prog.
END.

FIND FIRST mla-doc-pend-aprov WHERE mla-doc-pend-aprov.nr-trans = iNrTransacao NO-LOCK NO-ERROR.

FIND FIRST documento NO-LOCK NO-ERROR.
IF NOT AVAIL documento THEN
    RETURN "NOK":U.
```

Para o preenchimento dos campos que não vão em listas, é igual na listagem.

Lembrando que para separadores e o componente de lista não há valor para retornar neles, são apenas para diferenciar o *layout* ou tratar a estrutura de filhos.

```
FOR EACH tt-campo-valor EXCLUSIVE-LOCK:
CASE tt-campo-valor.id-campo:
    WHEN "doc-numero":U THEN
        ASSIGN tt-campo-valor.valor-campo = STRING(documento.num-doc).
    WHEN "doc-descricao":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.descricao.
    WHEN "doc-situacao":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.situacao.
    WHEN "doc-responsavel-prin":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.responsavel.
    WHEN "doc-valor":U THEN
        ASSIGN tt-campo-valor.valor-campo = STRING(documento.valor, ">>>>>>>>>9.99").
    WHEN "doc-narrativa":U THEN
        ASSIGN tt-campo-valor.valor-campo = documento.narrativa.
```

Já para o preenchimento de campos que vão dentro de listas sugere-se a seguinte estrutura.

Preencher as **variáveis** para indicar:

- Que há campos na lista – **l-existe-campo-xxx**;
- Para criar o campo filho X – **l-cria-filho-xxx**;
- Nome do campo filho X – **c-campo-filho-xxx**;

Lembrando que estas duas variáveis são *extent*, então para cada campo, deve-se usar uma posição. Por fim, deve-se eliminar o registro da *temp-table* **tt-campo-valor**. Isso é necessário, pois teremos que criar um registro deste componente para **cada linha** filha existente.

```
/* Campos da lista de itens */
WHEN "doc-item-codigo":U THEN DO:
    ASSIGN l-existe-campo-itens      = YES
           l-cria-filho-itens[1]    = YES
           c-campo-filho-itens[1]   = "doc-item-codigo":U.
    DELETE tt-campo-valor.
END.

WHEN "doc-item-desc":U THEN DO:
    ASSIGN l-existe-campo-itens      = YES
           l-cria-filho-itens[2]    = YES
           c-campo-filho-itens[2]   = "doc-item-desc":U.
    DELETE tt-campo-valor.
END.

/* Campos da lista de entregas */
WHEN "doc-entrega-seq":U THEN DO:
    ASSIGN l-existe-campo-entregas   = YES
           l-cria-filho-entregas[1] = YES
           c-campo-filho-entregas[1] = "doc-entrega-seq":U.
    DELETE tt-campo-valor.
END.

WHEN "doc-entrega-data":U THEN DO:
    ASSIGN l-existe-campo-entregas   = YES
           l-cria-filho-entregas[2] = YES
           c-campo-filho-entregas[2] = "doc-entrega-data":U.
    DELETE tt-campo-valor.
END.
```

O trecho de código a seguir, deve vir depois do **FOR EACH** da **tt-campo-valor**. Neste momento, precisamos preencher os campos das listas, criando um registro para cada linha e componente existente no modelo definido.

1. Inicializar o **contador da lista**, que será incrementado a **cada linha**. Neste caso, a cada item encontrado do documento.
2. No caso de haver mais um nível de lista dentro desta, é necessário eliminar o registro o componente de lista, pois será necessário criar um por linha de item. Definir o *buffer* no início do programa: **DEFINE BUFFER bf-tt-campo-valor FOR tt-campo-valor**.
3. Realizar a leitura da tabela filha, com base no pai. Passar por todos os campos da lista. Neste caso são dois. Se o campo em questão deve ser retornado, criar um registro na **tt-campo-valor**, se atentando para o **ID do campo, linha** e o **ID do pai**. Preencher o valor do campo conforme regras de negócio.
4. Trecho será explicado na próxima imagem.

```

IF l-existe-campo-itens THEN DO:
    ASSIGN i-cont-itens = 1.
FOR EACH bf-tt-campo-valor WHERE bf-tt-campo-valor.id-campo = "doc-list-entrega":U EXCLUSIVE-LOCK:
    DELETE bf-tt-campo-valor.
END.

FOR EACH item-doc WHERE item-doc.num-doc = documento.num-doc NO-LOCK:
    ASSIGN i-cont-campos-itens = 1.

    DO WHILE i-cont-campos-itens <= 2:
        IF l-cria-filho-itens[i-cont-campos-itens] THEN DO:

            CREATE tt-campo-valor.
            ASSIGN tt-campo-valor.id-campo      = c-campo-filho-itens[i-cont-campos-itens]
                   tt-campo-valor.linha        = i-cont-itens
                   tt-campo-valor.id-pai        = "doc-lista-itens":U.

            CASE c-campo-filho-itens[i-cont-campos-itens]:
                WHEN "doc-item-codigo":U THEN
                    ASSIGN tt-campo-valor.valor-campo = item-doc.cod-item.

                WHEN "doc-item-desc":U THEN
                    ASSIGN tt-campo-valor.valor-campo = item-doc.descricao.

            END CASE.
        END.
        ASSIGN i-cont-campos-itens = i-cont-campos-itens + 1.
    END.

    IF l-existe-campo-entregas THEN DO:
        <...>
    END.
    ASSIGN i-cont-itens = i-cont-itens + 1.
END.
END.

```

Sobre o preenchimento dos campos para a **lista de entregas**:

1. Se houverem campos a serem apresentados para a segunda lista, neste caso, de entregas, deve fazer basicamente o mesmo tratamento que a primeira lista para preencher os campos.
2. **Diferença:** Será necessário criar um componente de lista, para cada registro da lista anterior, neste caso, para cada item.
3. **Diferença:** Necessário preencher o campo de linha pai, com o número da linha pai. Neste caso, linha do item em questão.

```

IF 1-existe-campo-entregas THEN DO:
    ASSIGN i-cont-entregas = 1.
    FOR EACH entrega WHERE entrega.num-doc = item-doc.num-doc
        AND entrega.cod-item = item-doc.cod-item NO-LOCK:
            ASSIGN i-cont-campos-entregas = 1.

            FIND FIRST tt-campo-valor NO-LOCK
                WHERE tt-campo-valor.id-campo = "doc-list-entrega":U
                AND tt-campo-valor.linha = i-cont-itens NO-ERROR.

            IF NOT AVAIL tt-campo-valor THEN DO:
                CREATE tt-campo-valor.
                ASSIGN tt-campo-valor.id-campo = "doc-list-entrega":U
                tt-campo-valor.linha = i-cont-itens
                tt-campo-valor.id-pai = "doc-lista-itens":U.
            END.

            DO WHILE i-cont-campos-entregas <= 2:
                IF 1-cria-filho-entregas[i-cont-campos-entregas] THEN DO:

                    CREATE tt-campo-valor.
                    ASSIGN tt-campo-valor.id-campo = c-campo-filho-entregas[i-cont-campos-entregas]
                    tt-campo-valor.linha = i-cont-entregas
                    tt-campo-valor.id-pai = "doc-list-entrega":U
                    tt-campo-valor.linha-pai = i-cont-itens.

                    CASE c-campo-filho-entregas[i-cont-campos-entregas]:
                        WHEN "doc-entrega-seq":U THEN
                            ASSIGN tt-campo-valor.valor-campo = STRING(entrega.seq-entrega).

                        WHEN "doc-entrega-data":U THEN
                            ASSIGN tt-campo-valor.valor-campo = STRING(entrega.data, "99/99/9999").

                    END CASE.
                END.
                ASSIGN i-cont-campos-entregas = i-cont-campos-entregas + 1.
            END.
            ASSIGN i-cont-entregas = i-cont-entregas + 1.
        END.
    END.
END.

```

Caso nas *procedures* de retorno de campos tenha definido para mostrar campos genéricos do **MLA** é necessário o preenchimento deles.

```

RUN pi-preenche-detalle IN h-mla-prog (INPUT iNrTransacao,
                                         INPUT-OUTPUT TABLE tt-campo-valor).

```

Verificando o resultado do programa no **Configurador Visual – Lista:**

Configurador Visual M

TOTVS Construção e Projetos (Linha Datasul) 06.9.5554
ORACLE
Super Usuário
Atalho local: c:\dc117-ora
TOTVS S.A. - Ambiente de Desenvolvimento...
Sair

Aplicativo

CONFIGURAÇÕES DE LAYOUT

Configurações Visuais Aplicativo MLA

Configuração do aprovador

Configurações de layout

501 - Documento Específico

Voltar

Desfazer

Salvar

Você pode personalizar as informações conforme a necessidade do usuário. Será possível configurar os campos a serem apresentados no aplicativo, assim como o espaço ocupado (Tamanho da linha) dos mesmos.

Copiar layout

Redefinir padrão

Lista de Pendências

Detalhe das Pendências

CAMPO	Rótulo	Descrição	Visível	Tamanho da linha
	Nr	Número do Documento	☒	100%
	Desc	Descrição do Documento	☒	100%
	Sit	Situação do Documento	☒	100%

Configurador Visual M

TOTVS Construção e Projetos (Linha Datasul) 06.9.5554
ORACLE
Super Usuário
Atalho local: c:\dc117-ora
TOTVS S.A. - Ambiente de Desenvolvimento...
Sair

Aplicativo

CONFIGURAÇÕES DE LAYOUT

Configurações Visuais Aplicativo MLA

Configuração do aprovador

Configurações de layout

501 - Documento Específico

Voltar

Desfazer

Salvar

Você pode personalizar as informações conforme a necessidade do usuário. Será possível configurar os campos a serem apresentados no aplicativo, assim como o espaço ocupado (Tamanho da linha) dos mesmos.

Copiar layout

Redefinir padrão

Lista de Pendências

Detalhe das Pendências

CAMPO	Rótulo	Descrição	Visível	Tamanho da linha
	Resp	Responsável do Documento	☒	50%
	Valor	Valor do Documento	☒	50%
	Narrativa	Narrativa do Documento	☐	100%
	+ Adicionar rótulo	Descrição do Documento da Pendência de Aprovação	☐	100%

Verificando o resultado do programa no **Configurador Visual – Detalhe:**

Configurador Visual M

TOTVS

TOTVS Construção e Projetos (Linha Datasul) 06.9.5554

ORACLE

Super Usuário

Atalho local: c:\dic117-ora

TOTVS S.A. - Ambiente de Desenvolvimento...

Sair

Aplicativos

CONFIGURAÇÕES DE LAYOUT

Configurações Visuais Aplicativo MLA

Configuração do aprovador

Configurações de layout

501 - Documento Específico

Voltar

Desfazer

Salvar

Lista de Pendências

Detalhe das Pendências

CAMPO	Rótulo	Descrição	Visível	Tamanho da linha
	Número	Número do Documento	☒	100%
	Descrição	Descrição do Documento	☒	100%
	Situação	Situação do Documento	☒	100%
	Responsável Principal	Responsável Principal do Documento	☒	50%

Configurador Visual M

TOTVS

TOTVS Construção e Projetos (Linha Datasul) 06.9.5554

ORACLE

Super Usuário

Atalho local: c:\dic117-ora

TOTVS S.A. - Ambiente de Desenvolvimento...

Sair

Aplicativos

CONFIGURAÇÕES DE LAYOUT

Configurações Visuais Aplicativo MLA

Configuração do aprovador

Configurações de layout

501 - Documento Específico

Voltar

Desfazer

Salvar

CAMPO	Rótulo	Descrição	Visível	Tamanho da linha
	Valor	Valor do Documento	☒	50%

LISTA

Rótulo	Descrição	Visível	
Itens	Itens do documento	☒	Ocultar

CAMPO

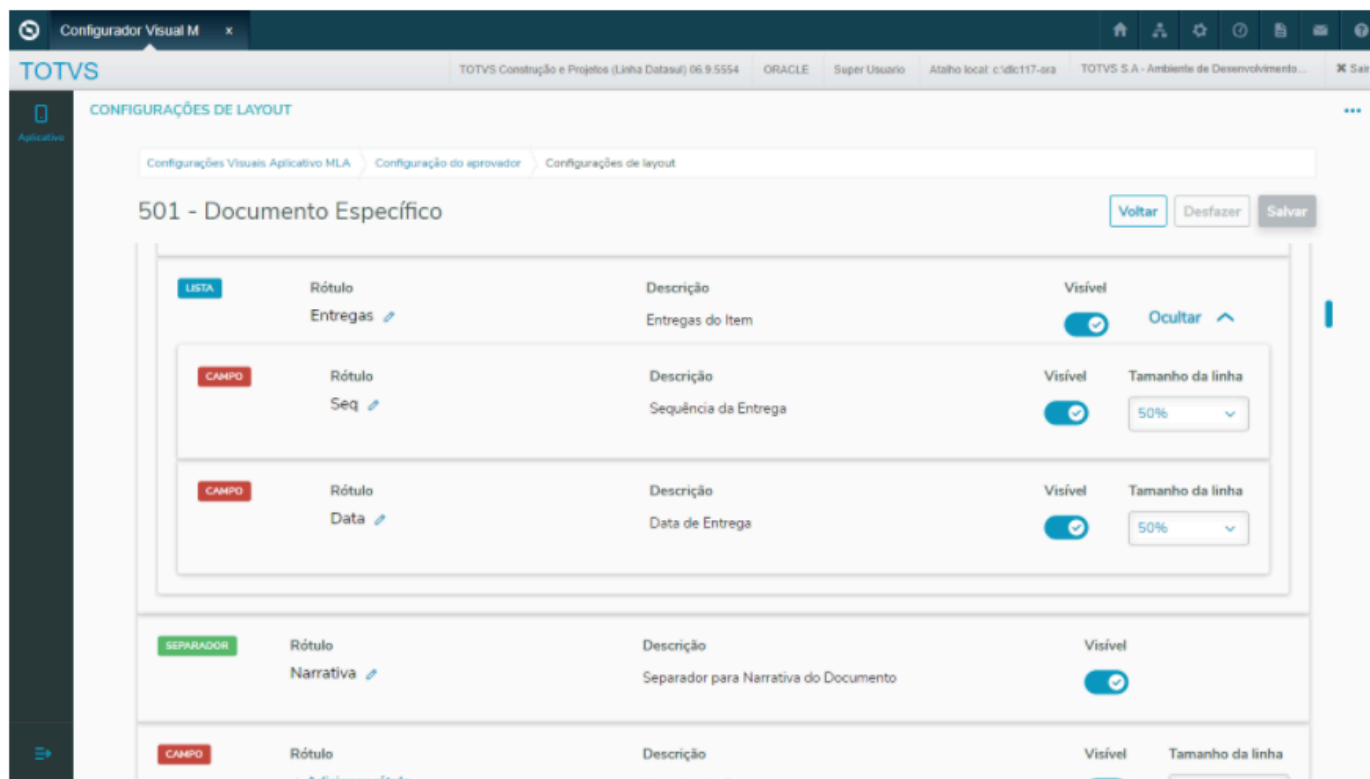
Rótulo	Descrição	Visível	Tamanho da linha
Código	Código do Item	☒	100%
Descrição	Descrição do Item	☒	100%

LISTA

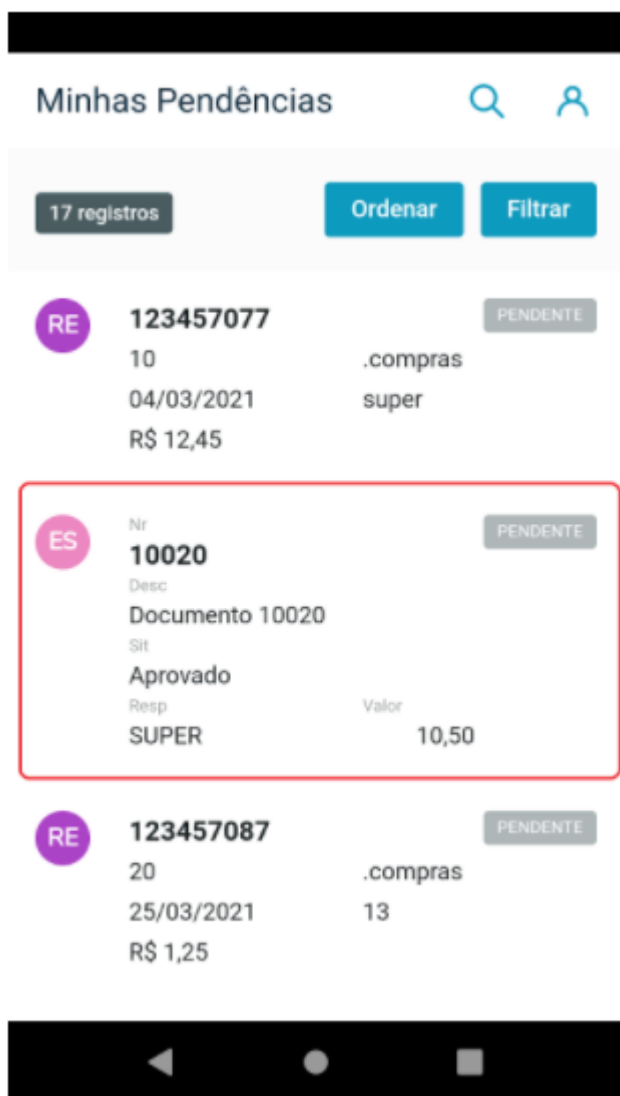
Rótulo	Descrição	Visível	
Entregas	Entregas do Item	☒	Ocultar

CAMPO

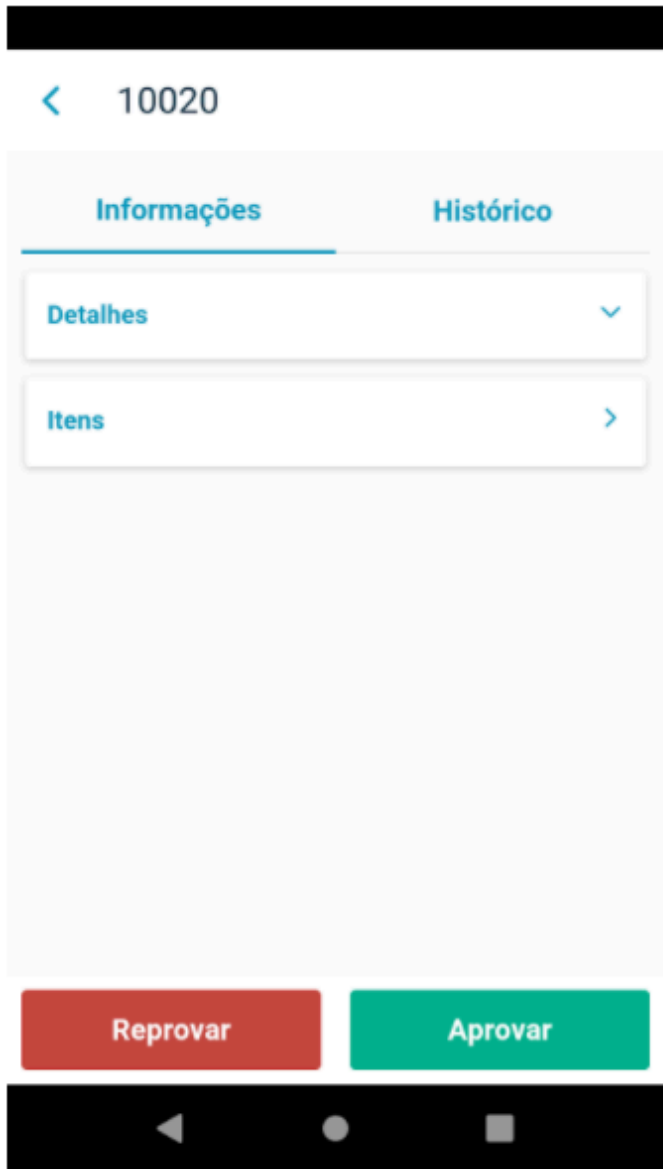
Rótulo	Descrição	Visível	Tamanho da linha
--------	-----------	---------	------------------

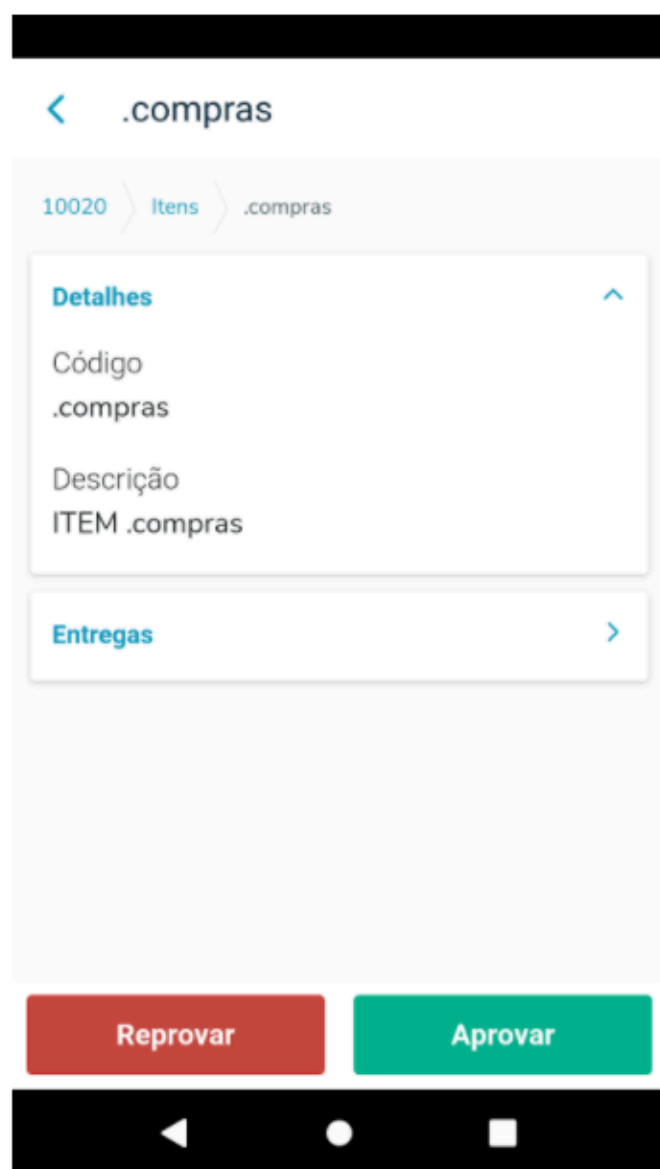
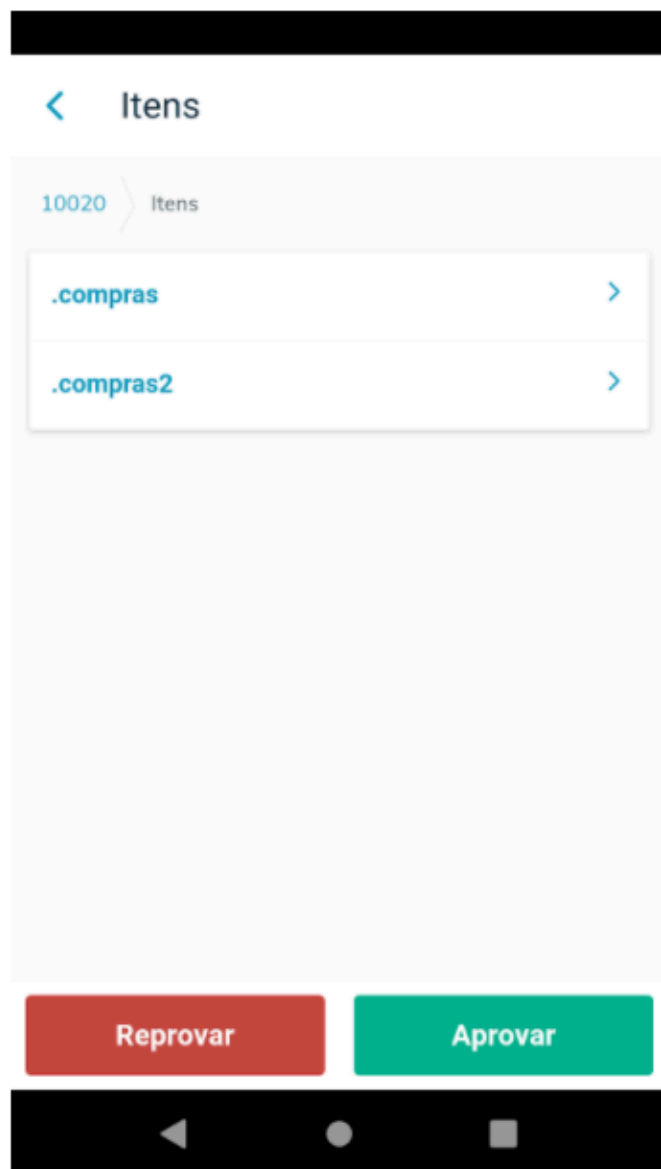


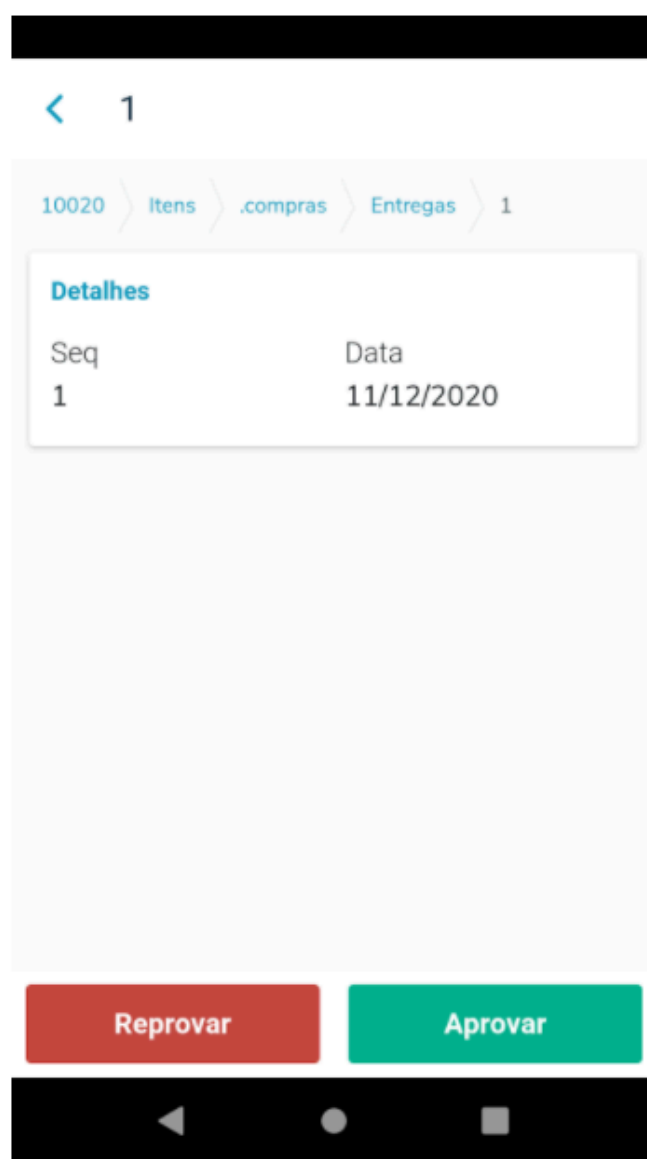
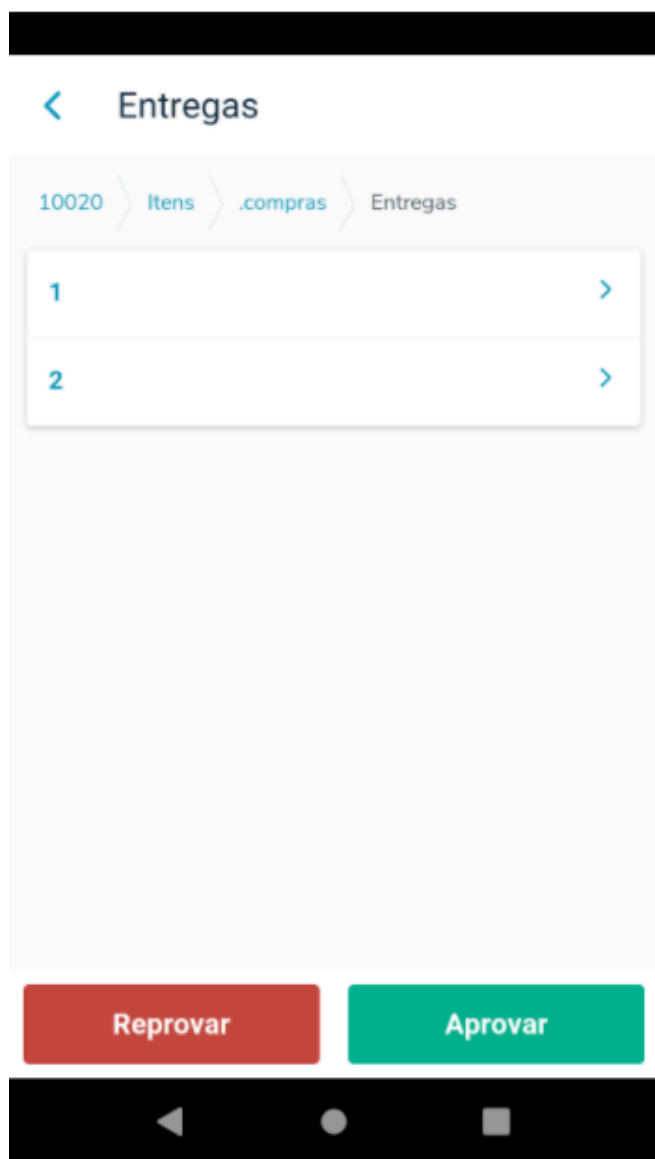
Verificando o resultado do programa no **APP – Lista:**



Verificando o resultado do programa no **APP – Detalhe:**







Saiba mais

Para os documentos padrões é importante que em cada *procedure* citada, seja incluso os pontos de **EPC - External Program Call** padronizados, para que seja possível **customizar** o documento. Para verificar quais pontos devem ser disponibilizados verificar a documentação Cross Segmentos - Linha Datasul - MLA - Customização de documento padrão disponibilizado no APP

Anexo

Para conferir um exemplo completo da **API**, veja o anexo abaixo.